Lesson 13: A Hit That Reacts

Unreal Engine (optional advanced track)

VIDEO GAME CREATION

This week: getting hit finally LOOKS like getting hit. A brief flash, then a small explosion, before the enemy disappears.

Open your own [YourName]Game project from Lesson 12 - don't start a new one.

Important: your score, lives, wave, and win/lose logic should all still work exactly as before. Today only changes how the enemy's removal LOOKS, not when your game-state actually changes.

Setup

1. Open your [YourName]Game project from Lesson 12.
2. Import enemy_invader_white.png (this lesson's Assets folder) into the Content Browser, then build M_Enemy_White the same way Lesson 4 built M_Enemy_Red - a new Material, Shading Model = Unlit, a Texture Sample using enemy_invader_white.png wired straight into Emissive Color. No Color parameter needed this time - unlike M_Enemy_Red, this Material exists purely to be swapped in as a whole for the flash, the same way M_Player_Red and friends were back in Lesson 2's Recolor.
3. Import explosion_placeholder.png into the Content Browser, then build M_Explosion: a new Material, Shading Model = Unlit, and Blend Mode = Masked (Details panel, right below Shading Model) - Masked lets parts of a texture render fully see-through instead of showing as a solid rectangle, which a plain Opaque Unlit material can't do. Add a Texture Sample using explosion_placeholder.png, wire its RGB output into Emissive Color and its Alpha output into Opacity Mask.
4. Create a new Actor Blueprint, BP_Explosion, using a Plane Static Mesh Component (Place Actors panel's Basic Shapes, same built-in set as Cube and Sphere - no download needed) with M_Explosion assigned to it. A Plane is a flat quad, the natural fit for showing a 2D sprite texture face-on to the top-down Orthographic Camera from Lesson 2, unlike a Cube or Sphere which would show the image wrapped oddly around a 3D volume.
5. In BP_Explosion's Event Graph: BeginPlay -> a Timeline (0.3 seconds) driving the Actor's Scale from small to large -> On Timeline Finished -> Destroy Actor - it cleans itself up automatically.

Level 1 - Minimal (MVP)

On every hit, the enemy briefly flashes, and a small independent explosion effect appears and cleans itself up. Score, lives, wave, and win/lose logic are all unaffected.

6. In HandleEnemyDestroyed (from Lesson 11/12), right after the existing sound/score updates, add Spawn Actor from Class(BP_Explosion) at the enemy's location - this happens immediately and independently of everything else.
7. Swap the enemy's own Material to M_Enemy_White with a Set Material node (Material Index 0) - the same real sprite-swap flash every other engine uses this week, just reached through Unreal's Material system - then add a Delay(0.1) node before the rest of the graph continues. There's no need to set it back afterward: the enemy is destroyed a moment later anyway.
8. Everything after the Delay (EnemiesRemaining update, the win-check nested Branch, Destroy Actor on the enemy) stays exactly as it was - just running a tenth of a second later in the same graph.
9. Play in Editor and play through a full round - each hit should flash the enemy briefly, and a small shape should scale up and disappear at that spot.

```
Custom Event HandleEnemyDestroyed (revised) ->
  Play Sound 2D(sfx_hit.wav)
  Spawn Actor from Class(BP_Explosion, Transform = enemy's current
Transform)
  Set Material(Self, Material Index 0, M_Enemy_White)
  Delay(0.1)
  EnemiesRemaining = EnemiesRemaining - 1
  Score = Score + 10
  Call UpdateHUD(HUDWidget, Score, Lives, Wave)
  Branch(EnemiesRemaining <= 0) -> True:
    Branch(Wave >= TotalWaves) -> True:
      GameActive = false
      Play Sound 2D(sfx_win.wav)
      Call SetStatus(HUDWidget, "You Win! Press R to Restart")
    False:
      Wave = Wave + 1
      EnemiesRemaining = 5
      Play Sound 2D(sfx_wave.wav)
      Call UpdateHUD(HUDWidget, Score, Lives, Wave)
      Call SpawnWave
  Destroy Actor (the enemy)
```

Alternate Path: C++ (code, optional advanced track)

The same flash-then-explode idea works in C++ using a Timer Handle in place of a Blueprint Delay node.

10. Spawn ABPExplosion and call SetMaterial() on the enemy's mesh component to swap in M_Enemy_White at the top of your kill-handling function, then use GetWorldTimerManager().SetTimer() with a 0.1-second delay to call a second function containing the rest of the existing kill logic (EnemiesRemaining update, win-check, Destroy).

This mirrors the Blueprint path's Delay node exactly - same pause, just implemented with a Timer Handle instead.

Level 2 - Complete (Alpha/Beta)

The flash and explosion play correctly on every single hit, with no leftover explosions, no enemies stuck mid-flash, and no change to score/lives/wave timing.

- Play a full round: every single hit shows a flash and an explosion, with no enemy left stuck mid-flash.
- Confirm score, wave-clear, and win/lose all still happen at exactly the moments they did last week - Delay only pauses this ONE execution wire, nothing else in the level pauses.
- Confirm BP_Explosion Actors never pile up in the World Outliner - each one destroys itself via its own Timeline, with no help from the enemy.

Level 3 - Refined (Ready to Publish) - pick any that interest you

- Visual: instead of an instant Set Material snap, build M_Enemy_Flash with both enemy_invader_red.png and enemy_invader_white.png as separate Texture Samples, blended by a Lerp node driven by a scalar Blend Parameter, and animate that parameter from 1 (white) back down to 0 (red) across a Timeline instead of relying on the Delay - a genuine cross-fade instead of a hard cut.
- Visual: add a second Timeline track to BP_Explosion driving a new Opacity scalar Parameter on M_Explosion from 1 down to 0 as the shape scales up, so it fades out instead of popping away when destroyed - this needs M_Explosion's Blend Mode changed from Masked to Translucent first (Masked can only fully show or fully hide a pixel, not fade it), set through a Dynamic Material Instance the same way BP_Enemy's Color parameter works back in Lesson 4.
- Structure: notice BP_Explosion knows nothing about score, lives, or waves - it's a fully self-contained Actor, the same discipline as this course's bullets and enemies.

If Something's Wrong

- Enemy vanishes instantly, no flash visible: confirm the Delay(0.1) node was actually added, and Destroy Actor comes AFTER it, not before.
- Enemy renders solid black on the flash instead of white: confirm M_Enemy_White exists as its own separate Material (Texture Sample -> Emissive Color, with enemy_invader_white.png actually assigned), and that Set Material is pointing at M_Enemy_White, not accidentally back at M_Enemy_Red.
- Explosion never appears: confirm BP_Explosion is actually referenced correctly in Spawn Actor from Class, and that its Timeline is set to auto-play on BeginPlay.
- Explosion appears as a plain gray square instead of the burst shape, or doesn't appear at all: confirm M_Explosion's Texture Sample has both its RGB output wired to Emissive Color AND its Alpha output wired to Opacity Mask, that explosion_placeholder.png is actually assigned as its Texture, and that the Material's Blend Mode is actually set to Masked, not left on the default Opaque.
- Explosion doesn't clean itself up: confirm its Timeline's Finished pin is actually wired to Destroy Actor.

Want More?

dev.epicgames.com/community/learning, Student Resource List > Unreal